

Findings for http://vulnerable-target:80

Below is a concise “check-list” of the most common vulnerabilities that tend to appear in PHP-driven, container-ized web applications. The list is organized by the OWASP Top 10 (2021) categories, with brief explanations of *why* each issue shows up in PHP projects and *what* you should look for during a manual or automated review. It is intended for a legitimate security assessment (e.g., a code-review, configuration audit, or penetration-test that you have permission to perform); no exploitation instructions are included.

1. Injection

Vulnerability	Typical PHP cause	What to look for
SQL Injection (SQLi)	Direct interpolation of user input into SQL strings, e.g. <code>mysqli_query(\$db, "SELECT * FROM users WHERE id=\$id")</code> . Use of outdated APIs (<code>mysql_*</code>) that lack prepared-statement support.	• Queries built with string concatenation or <code>sprintf</code>. • Missing <code>mysqli::prepare</code> / PDO prepared statements. • Unsanitized <code>\$_GET</code>, <code>\$_POST</code>, <code>\$_COOKIE</code>, <code>\$_REQUEST</code> used in <code>WHERE</code>, <code>ORDER BY</code>, or dynamic table/column names.
Command Injection	Passing user data to shell commands (<code>exec()</code> , <code>`` ``</code> , <code>system()</code> , <code>passthru()</code> , <code>proc_open()</code>).	• Calls to <code>exec</code>, <code>shell_exec</code>, <code>system</code>, <code>popen</code>, <code>proc_open</code> where arguments include <code>\$_GET/POST/REQUEST</code>. • Use of <code>eval()</code> or backticks with interpolated variables.
LDAP / XPath Injection	Building LDAP filters or XPath queries with raw input.	• <code>ldap_search()</code> or <code>DOMXPath->query()</code> that embed user data without escaping.
NoSQL Injection (MongoDB, etc.)	Direct insertion of request parameters into query arrays.	• <code>\$collection->find(['username' => \$_GET['u']]);</code> without validation.

2. Broken Authentication & Session Management

Vulnerability	Typical PHP cause	What to look for
Weak password storage	Plain-text, MD5, SHA1, or unsalted hashes.	• <code>password_hash</code> not used; look for <code>md5()</code>, <code>sha1()</code>, <code>crypt()</code> without a strong salt.
Session fixation / hijacking	Fixed session ID, lack of <code>session_regenerate_id()</code> , insecure cookie attributes.	• No <code>session_start()</code> parameters (<code>cookie_httponly</code>, <code>cookie_secure</code>).

Vulnerability	Typical PHP cause	What to look for
		Session ID taken from URL (<code>PHPSESSID</code> in GET).
Brute-force login	No rate limiting, CAPTCHA, or account lockout.	Login endpoint that accepts unlimited attempts.
Insecure “remember-me” tokens	Tokens stored in clear text or predictable values.	Cookies that contain user IDs or passwords.
Improper logout	Session not destroyed, only UI redirect.	<code>logout.php</code> that only <code>unset(\$_SESSION)</code> without <code>session_destroy()</code>.

3. Sensitive Data Exposure

Vulnerability	Typical PHP cause	What to look for
Plain-text transmission	Using HTTP (not HTTPS) or not forcing <code>Strict-Transport-Security</code> .	Application reachable on port 80; check <code>.htaccess</code> or Docker config for redirects to HTTPS.
Improper error handling	Stack traces, database errors, or configuration data displayed to users.	<code>display_errors = On</code> in <code>php.ini</code> or <code>ini_set('display_errors', 1)</code>.
Insecure storage	Secrets in source code, config files, or version-controlled files (<code>.env</code> , <code>.git</code>).	Search the repo for patterns: <code>password=</code>, <code>API_KEY</code>, <code>JWT_SECRET</code>.
Cacheable sensitive pages	Missing <code>Cache-Control: no-store</code> on admin pages.	Look at response headers for sensitive URLs.
Weak cryptography	Custom encryption with <code>mcrypt</code> or <code>openssl_encrypt</code> using static IVs/keys.	Hard-coded keys, predictable IVs.

4. XML External Entity (XXE)

Vulnerability	Typical PHP cause	What to look for
XXE	<code>simplexml_load_string</code> , <code>DOMDocument::loadXML</code> , or <code>XMLReader</code> with external entity loading enabled.	<code>libxml_disable_entity_loader(false)</code> or missing <code>LIBXML_NOENT</code>.
XML injection	Unvalidated XML used to influence application logic.	Direct insertion of user data into XML payloads.

5. Broken Access Control

Vulnerability	Typical PHP cause	What to look for
Horizontal privilege escalation	Relying solely on client-side checks (hidden fields, JavaScript) or on the presence of an ID parameter.	URLs like <code>profile.php?id=123</code> where any logged-in user can change the <code>id</code>.
Vertical privilege escalation	Admin functions accessible without role verification.	Missing <code>if (\$_SESSION['role'] === 'admin')</code> guard.
Insecure direct object references (IDOR)	File paths/DB keys taken from request without validation.	<code>download.php?file=../../etc/passwd</code>.
Missing CSRF protection	No anti-CSRF token on state-changing POST/PUT/DELETE actions.	Forms without hidden token, or token not verified server side.
CORS misconfiguration	<code>Access-Control-Allow-Origin: *</code> on authenticated endpoints.	Check response headers for wide-open CORS.

6. Security Misconfiguration

Vulnerability	Typical PHP cause	What to look for
Default/weak credentials	Default MySQL root password, admin/guest accounts left unchanged.	Docker <code>environment</code> variables (<code>MYSQL_ROOT_PASSWORD= empty</code>).
Exposed debug tools	PHPInfo (<code>phpinfo.php</code>) or web-based admin panels left public.	Files named <code>phpinfo.php</code>, <code>info.php</code>, <code>adminer.php</code>.
Directory listing	Missing <code>Options -Indexes</code> in Apache/Nginx config.	Attempt to browse <code>/uploads/</code> or <code>/src/</code>.
Unrestricted file upload	No MIME/type validation, no sanitization of filenames, no storage outside web root.	Upload endpoints that accept <code>\$_FILES</code> and move them with <code>move_uploaded_file</code> to a public directory.
Out-of-date components	PHP version < 7.4, old libraries (e.g., <code>symfony/Yaml</code> with known CVEs).	<code>composer.lock</code> versions, container base-image tags.
Improper permissions	Files/directories world-writable (<code>chmod 777</code>).	Check Dockerfile <code>RUN chmod</code> statements.

7. Cross-Site Scripting (XSS)

Variant	Typical PHP cause	What to look for
Reflected XSS	Echoing request parameters without escaping (e.g., <code>echo \$_GET['q'];</code>).	• <code><?= \$_GET['...'] ?></code> or <code>print \$_POST['...']</code> directly into HTML.
Stored XSS	Persisting user input (comments, profiles) and later rendering it raw.	• Database inserts that store raw HTML.
DOM-based XSS	Server sends JSON that is later inserted into the DOM via unsafe JavaScript.	• API responses with unescaped values used by client scripts.
Mitigations to verify	Proper use of <code>htmlspecialchars(\$data, ENT_QUOTES</code>	<code>ENT_SUBSTITUTE, 'UTF-8')</code> or templating engines that auto-escape (Twig, Blade).

8. Insecure Deserialization

Vulnerability	Typical PHP cause	What to look for
PHP Object Injection	<code>unserialize()</code> on data that an attacker can control (cookies, hidden fields, API parameters).	• Calls to <code>unserialize(\$_GET['data'])</code>, <code>unserialize(\$_SESSION['payload'])</code>.
Magic-method abuse (<code>__wakeup</code> , <code>__destruct</code>)	Attacker-controlled objects can trigger file writes, command execution.	• Classes with dangerous magic methods; search for <code>function __wakeup</code> or <code>function __destruct</code> that interact with the filesystem or exec.

9. Using Components with Known Vulnerabilities

What to check	Why it matters
Composer dependencies – run <code>composer audit</code> or check <code>composer.lock</code> against the OSS Index / Snyk database.	
Docker base images – verify that the image tag is not an outdated distribution (e.g., <code>php:7.2-apache</code>).	
PHP extensions – outdated versions of <code>imagick</code> , <code>gd</code> , <code>openssl</code> , etc., can have CVEs.	

10. Insufficient Logging & Monitoring

Vulnerability	Typical PHP cause	What to look for
No security-relevant logs	Application only logs generic messages, or logs are written to a world-writable file.	• Search for <code>error_log</code>, <code>syslog</code>, or custom logging utilities.
Log injection	User data written to logs without sanitization (e.g., <code>error_log(\$_GET['msg'])</code>).	• Potential to forge log entries or break log parsers.

Vulnerability	Typical PHP cause	What to look for
Lack of alerting	No integration with container orchestration (Kubernetes) or host-level SIEM.	• Review Docker/Compose files for log drivers.

How to Use This List During Your Review

1. Static Code Review

- Grep the source tree for risky functions (`eval`, `exec`, `system`, `passthru`, ```, `unserialize`, `mysql_`, `session_start`, `setcookie`, `file_put_contents`, etc.).
- Examine every place those functions interact with user-controlled data (`$_GET`, `$_POST`, `$_REQUEST`, `$_COOKIE`, `$_FILES`, environment variables).

2. Configuration Review

- Check `php.ini` (or `docker-php-ini` overrides) for `display_errors`, `expose_php`, `log_errors`, `session.cookie_secure`, `session.cookie_httponly`.
- Inspect the web server (Apache/Nginx) config inside the container for directory listings, CORS headers, HSTS, and TLS settings.

3. Dependency Audit

- Run `composer audit` inside the container or on the host copy of the repo.
- List OS packages (`apt list --installed` or `apk info`) and compare with known CVE feeds.

4. Dynamic Testing (Allowed by Scope)

- Use a scanner (e.g., OWASP ZAP, Nikto, or Burp Suite) to surface obvious injection/ XSS/CSRF issues.
- Manually test authentication flows for session fixation, CSRF, rate limiting.

5. Container-Specific Checks

- Ensure the container runs as a non-root user (`USER` directive).
- Verify that only necessary ports are exposed and that file system permissions are restrictive (`chmod 640` for config, `chmod 750` for code).
- Look for secret leakage via environment variables in `docker-compose.yml` or Kubernetes manifests.

Quick "Cheat-Sheet" of Red-Flag Code Patterns

```
// 1. Direct interpolation → SQLi
$sql = "SELECT * FROM users WHERE email = '{$_GET['email']}'";

// 2. Unsanitized command execution → Command injection
system("ping -c 4 " . $_GET['host']);

// 3. Unescaped output → XSS
```

```
echo "<div>{$_POST['comment']}
```

Whenever you spot one of these (or a variant), flag it for deeper analysis—add proper input validation, prepared statements, output escaping, secure configuration, etc.

Bottom line: PHP’s flexibility makes it easy to slip insecure practices into the codebase, especially when developers rely on quick string concatenation or turn off error reporting to “keep the site pretty”. By systematically walking through the categories above—focusing on input handling, authentication, configuration, and component versions—you’ll be able to uncover the most common—and often the most exploitable—issues in a containerized PHP web app. Good luck with your review, and stay within your authorized scope!

Findings for http://vulnerable-target:80

Below is a concise “check-list” of the most common vulnerabilities that tend to appear in PHP-driven, container-ized web applications. The list is organized by the OWASP Top 10 (2021) categories, with brief explanations of *why* each issue shows up in PHP projects and *what* you should look for during a manual or automated review. It is intended for a legitimate security assessment (e.g., a code-review, configuration audit, or penetration-test that you have permission to perform); no exploitation instructions are included.

1. Injection

Vulnerability	Typical PHP cause	What to look for
SQL Injection (SQLi)	Direct interpolation of user input into SQL strings, e.g. ``php	
// 1. Direct interpolation → SQLi		

Vulnerability	Typical PHP cause	What to look for
	<pre>\$sql = "SELECT * FROM users WHERE email = '{\$_GET['email']}'; // 2. Unsanitized command execution → Command injection system("ping -c 4 " . \$_GET['host']); // 3. Unescaped output → XSS echo " {\$_POST['comment']} "; // 4. unserialize on user data → PHP object injection \$obj = unserialize(\$_COOKIE['payload']); // 5. Deprecated mysql_* API \$result = mysql_query("SELECT * FROM products"); // 6. File upload without validation move_uploaded_file(\$_FILES['file']['tmp_name'], "uploads/" . \$_FILES['file'] ['name']); // 7. Missing CSRF token check if (\$_SERVER['REQUEST_METHOD'] === 'POST') { // process form without verifying token } // 8. Session fixation session_id(\$_GET['sid']); session_start();</pre>	

Whenever you spot one of these (or a variant), flag it for deeper analysis—add proper input validation, prepared statements, output escaping, secure configuration, etc.

****Bottom line:**** PHP's flexibility makes it easy to slip insecure practices into the codebase, especially when developers rely on quick string concatenation or turn off error reporting to "keep the site pretty". By systematically walking through the categories above—focusing on input handling, authentication, configuration, and component versions—you'll be able to uncover the most common—and often the most exploitable—issues in a containerized PHP web app. Good luck with your review, and stay within your authorized scope!

Findings for http://vulnerable-target:80
Below is a concise "check-list" of the most common vulnerabilities that tend to appear in PHP-driven, container-ized web applications. The list is organized by the OWASP Top 10 (2021) categories, with brief explanations of **why** each issue shows up in PHP projects and **what** you should look for during a manual or automated review. It is intended for a legitimate security assessment (e.g., a code-review, configuration audit, or penetration-test that you have permission to perform); no exploitation instructions are included.

1. Injection

Vulnerability	Typical PHP cause	What to look for
SQL Injection (SQLi) Direct interpolation of user input into SQL strings, e.g. <code>`mysqli_query(\$db, "SELECT * FROM users WHERE id=\$id")`</code> . Use of outdated APIs (<code>`mysql_*`</code>) that lack prepared-statement support. • Queries built with string concatenation or <code>`sprintf`</code> . • Missing <code>`mysqli::prepare`</code> / PDO prepared statements. • Unsanitized <code>`\$_GET`</code> , <code>`\$_POST`</code> , <code>`\$_COOKIE`</code> , <code>`\$_REQUEST`</code> used in <code>`WHERE`</code> , <code>`ORDER BY`</code> , or dynamic table/column names.		
Command Injection Passing user data to shell commands (<code>`exec()`</code> , <code>` `` `` ``</code> , <code>`system()`</code> , <code>`passthru()`</code> , <code>`proc_open()`</code>). • Calls to <code>`exec`</code> , <code>`shell_exec`</code> , <code>`system`</code> , <code>`popen`</code> , <code>`proc_open`</code> where arguments include <code>`\$_GET/POST/REQUEST`</code> . • Use of <code>`eval()`</code> or backticks with interpolated variables.		
LDAP / XPath Injection Building LDAP filters or XPath queries with raw input. • <code>`ldap_search()`</code> or <code>`DOMXPath->query()`</code> that embed user data without escaping.		
NoSQL Injection (MongoDB, etc.) Direct insertion of request parameters into query arrays. • <code>`\$collection->find(['username' => \$_GET['u']]);`</code> without validation.		

2. Broken Authentication & Session Management

Vulnerability	Typical PHP cause	What to look for
Weak password storage Plain-text, MD5, SHA1, or unsalted hashes. • <code>`password_hash`</code> not used; look for <code>`md5()`</code> , <code>`sha1()`</code> , <code>`crypt()`</code> without a strong salt.		
Session fixation / hijacking Fixed session ID, lack of <code>`session_regenerate_id()`</code> , insecure cookie attributes. • No <code>`session_start()`</code> parameters (<code>`cookie_httponly`</code> , <code>`cookie_secure`</code>). • Session ID taken from URL (<code>`PHPSESSID`</code> in GET).		
Brute-force login No rate limiting, CAPTCHA, or account lockout. • Login endpoint that accepts unlimited attempts.		
Insecure “remember-me” tokens Tokens stored in clear text or predictable values. • Cookies that contain user IDs or passwords.		
Improper logout Session not destroyed, only UI redirect. • <code>`logout.php`</code> that only <code>`unset(\$_SESSION)`</code> without <code>`session_destroy()`</code> .		

3. Sensitive Data Exposure

Vulnerability	Typical PHP cause	What to look for
Plain-text transmission Using HTTP (not HTTPS) or not forcing <code>`Strict-Transport-Security`</code> . • Application reachable on port 80; check <code>`.htaccess`</code> or Docker config for redirects to HTTPS.		
Improper error handling Stack traces, database errors, or configuration data displayed to users. • <code>`display_errors = On`</code> in <code>`php.ini`</code> or <code>`ini_set('display_errors', 1)`</code> .		
Insecure storage Secrets in source code, config files, or		

version-controlled files (`.env`, `.git`). | • Search the repo for patterns: `password=`, `API\_KEY`, `JWT\_SECRET`. |
 | ****Cacheable sensitive pages**** | Missing `Cache-Control: no-store` on admin pages. | • Look at response headers for sensitive URLs. |
 | ****Weak cryptography**** | Custom encryption with `mcrypt` or `openssl\_encrypt` using static IVs/keys. | • Hard-coded keys, predictable IVs. |

4. XML External Entity (XXE)

Vulnerability	Typical PHP cause	What to look for
XXE	`simplexml_load_string`, `DOMDocument::loadXML`, or `XMLReader` with external entity loading enabled.	• `libxml_disable_entity_loader(false)` or missing `LIBXML_NOENT`.
XML injection	Unvalidated XML used to influence application logic.	• Direct insertion of user data into XML payloads.

5. Broken Access Control

Vulnerability	Typical PHP cause	What to look for
Horizontal privilege escalation	Relying solely on client-side checks (hidden fields, JavaScript) or on the presence of an ID parameter.	• URLs like `profile.php?id=123` where any logged-in user can change the `id`.
Vertical privilege escalation	Admin functions accessible without role verification.	• Missing `if (\$_SESSION['role']=='admin')` guard.
Insecure direct object references (IDOR)	File paths/DB keys taken from request without validation.	• `download.php?file=../../etc/passwd`.
Missing CSRF protection	No anti-CSRF token on state-changing POST/PUT/DELETE actions.	• Forms without hidden token, or token not verified server side.
CORS misconfiguration	`Access-Control-Allow-Origin: *` on authenticated endpoints.	• Check response headers for wide-open CORS.

6. Security Misconfiguration

Vulnerability	Typical PHP cause	What to look for
Default/weak credentials	Default MySQL root password, admin/guest accounts left unchanged.	• Docker `environment` variables (`MYSQL_ROOT_PASSWORD=` empty).
Exposed debug tools	PHPInfo (`phpinfo.php`) or web-based admin panels left public.	• Files named `phpinfo.php`, `info.php`, `adminer.php`.
Directory listing	Missing `Options -Indexes` in Apache/Nginx config.	• Attempt to browse `/uploads/` or `/src/`.
Unrestricted file upload	No MIME/type validation, no sanitization of filenames, no storage outside web root.	• Upload endpoints that accept `\$_FILES` and move them with `move_uploaded_file` to a public directory.

```
| **Out-of-date components** | PHP version < 7.4, old libraries (e.g.,
`symfony/Yaml` with known CVEs). | • `composer.lock` versions, container
base-image tags. |
| **Improper permissions** | Files/directories world-writable (`chmod 777`). | •
Check Dockerfile `RUN chmod` statements. |
```

7. Cross-Site Scripting (XSS)

```
| Variant | Typical PHP cause | What to look for |
|-----|-----|-----|
| **Reflected XSS** | Echoing request parameters without escaping (e.g., `echo
$_GET['q'];`). | • `<<?= $_GET['...'] ?>` or `print $_POST['...']` directly into
HTML. |
| **Stored XSS** | Persisting user input (comments, profiles) and later rendering
it raw. | • Database inserts that store raw HTML. |
| **DOM-based XSS** | Server sends JSON that is later inserted into the DOM via
unsafe JavaScript. | • API responses with unescaped values used by client scripts.
|
| **Mitigations to verify** | Proper use of `htmlspecialchars($data, ENT_QUOTES |
ENT_SUBSTITUTE, 'UTF-8')` or templating engines that auto-escape (Twig, Blade). |
• Look for consistent escaping functions. |
```

8. Insecure Deserialization

```
| Vulnerability | Typical PHP cause | What to look for |
|-----|-----|-----|
| **PHP Object Injection** | `unserialize()` on data that an attacker can control
(cookies, hidden fields, API parameters). | • Calls to
`unserialize($_GET['data'])`, `unserialize($_SESSION['payload'])`. |
| **Magic-method abuse** (`__wakeup`, `__destruct`) | Attacker-controlled objects
can trigger file writes, command execution. | • Classes with dangerous magic
methods; search for `function __wakeup` or `function __destruct` that interact
with the filesystem or exec. |
```

9. Using Components with Known Vulnerabilities

```
| What to check | Why it matters |
|-----|-----|
| **Composer dependencies** - run `composer audit` or check `composer.lock`
against the **OSS Index / Snyk** database. |
| **Docker base images** - verify that the image tag is not an outdated
distribution (e.g., `php:7.2-apache`). |
| **PHP extensions** - outdated versions of `imagick`, `gd`, `openssl`, etc., can
have CVEs. |
```

10. Insufficient Logging & Monitoring

Vulnerability	Typical PHP cause	What to look for
No security-relevant logs	Application only logs generic messages, or logs are written to a world-writable file.	• Search for <code>`error_log`</code> , <code>`syslog`</code> , or custom logging utilities.
Log injection	User data written to logs without sanitization (e.g., <code>`error_log(\$_GET['msg'])`</code>).	• Potential to forge log entries or break log parsers.
Lack of alerting	No integration with container orchestration (Kubernetes) or host-level SIEM.	• Review Docker/Compose files for log drivers.

How to Use This List During Your Review

1. **Static Code Review**

- Grep the source tree for risky functions (``eval``, ``exec``, ``system``, ``passthru``, ``__``, ``unserialize``, ``mysql_``, ``session_start``, ``setcookie``, ``file_put_contents``, etc.).
- Examine every place those functions interact with user-controlled data (``$_GET``, ``$_POST``, ``$_REQUEST``, ``$_COOKIE``, ``$_FILES``, environment variables).

2. **Configuration Review**

- Check ``php.ini`` (or ``docker-php-ini`` overrides) for ``display_errors``, ``expose_php``, ``log_errors``, ``session.cookie_secure``, ``session.cookie_httponly``.
- Inspect the web server (Apache/Nginx) config inside the container for directory listings, CORS headers, HSTS, and TLS settings.

3. **Dependency Audit**

- Run ``composer audit`` inside the container or on the host copy of the repo.
- List OS packages (``apt list --installed`` or ``apk info``) and compare with known CVE feeds.

4. **Dynamic Testing (Allowed by Scope)**

- Use a scanner (e.g., OWASP ZAP, Nikto, or Burp Suite) to surface obvious injection/ XSS/CSRF issues.
- Manually test authentication flows for session fixation, CSRF, rate limiting.

5. **Container-Specific Checks**

- Ensure the container runs as a non-root user (``USER`` directive).
- Verify that only necessary ports are exposed and that file system permissions are restrictive (``chmod 640`` for config, ``chmod 750`` for code).
- Look for secret leakage via environment variables in ``docker-compose.yml`` or Kubernetes manifests.

Quick “Cheat-Sheet” of Red-Flag Code Patterns

```
```php
// 1. Direct interpolation → SQLi
$sql = "SELECT * FROM users WHERE email = '{$_GET['email']}'";
```

```
// 2. Unsanitized command execution → Command injection
system("ping -c 4 " . $_GET['host']);

// 3. Unescaped output → XSS
echo "<div>{$_POST['comment']}
```

Whenever you spot one of these (or a variant), flag it for deeper analysis—add proper input validation, prepared statements, output escaping, secure configuration, etc.

---

**Bottom line:** PHP’s flexibility makes it easy to slip insecure practices into the codebase, especially when developers rely on quick string concatenation or turn off error reporting to “keep the site pretty”. By systematically walking through the categories above—focusing on input handling, authentication, configuration, and component versions—you’ll be able to uncover the most common—and often the most exploitable—issues in a containerized PHP web app. Good luck with your review, and stay within your authorized scope!

---